Reproducibility in Scientific Software Repositories and the Reusable Execution Environment

Anh Duc Vu, Christoph Blessing, Ana Dragoljic, Michael Goedicke

Sharing Code ≠ Sharing Science

Software Decay

Code that worked at publication often fails 6 months later due to silent breakage as the ecosystem evolves around it.

Undocumented Environments

The environment, OS version, compiler flags, libraries, runtime configuration and hardware are often undocumented.

Non-Determinism

Sources of randomness like parallel executions lead to non-determinism. The **same code may produce different results** across runs.

"No Data Without Software" — NFDIxCS

The **FAIR principles** of research data need to also apply to the software that generates and interprets it. Research management must treat the **software and its execution environment as a primary research artifact**, not an afterthought.

The Reproducibility Crisis — By The Numbers

70%

of 1,576 researchers **failed to reproduce** a published result

Nature survey — Baker 2016

13.7%

of Jupyter notebooks **declared their dependencies** — installation failure rates were **61–67%**

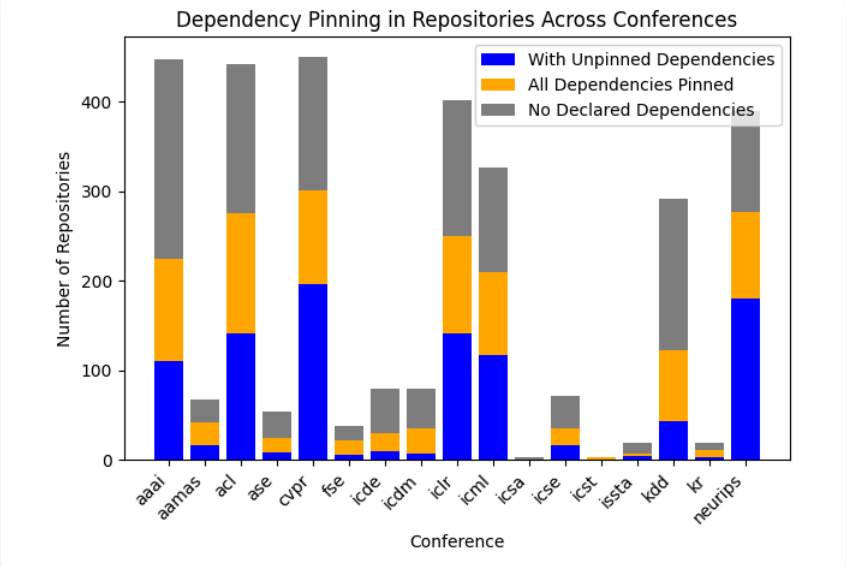
Pimentel et al., MSR 2019

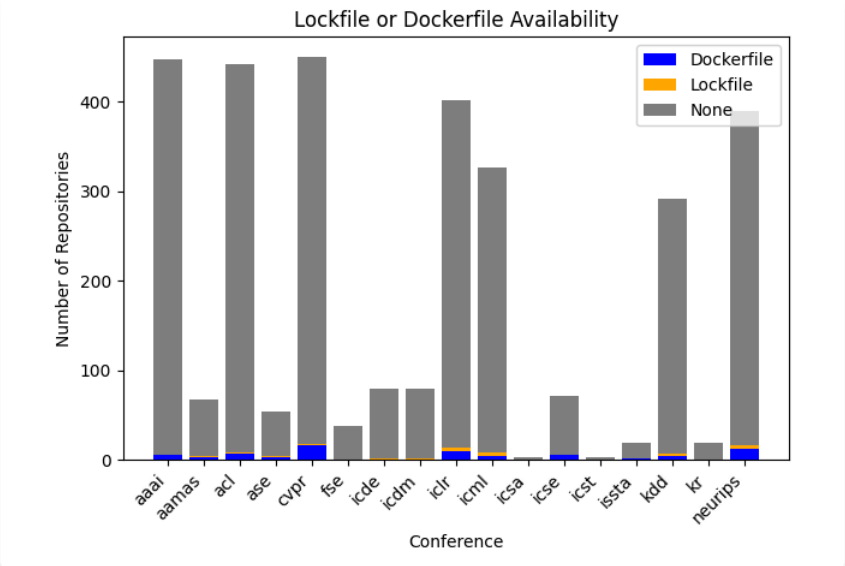
87.6% of biomedical notebooks threw exceptions on re-execution. The format designed for reproducibility fails in practice.

Samuel & Mietchen, GigaScience 2024

Dependency Management in Scientific Software Github Repositories

- Conducted a **GitHub search** spanning **2020–2025** for research software projects
- Targeted Python repositories from popular CS conferences: **ICSE, ICML, NeurIPS, ASE**, and others
- Retrieved using the query `conf_name + year + language:python`





Reproducibility Levels

Reproducibility Levels

LVL 01

Natural language only (README)

```
"This project requires python 3.10 and numpy."
```

❌ Likely unprecise and incomplete.
✅ Create a formal dependency file.

Reproducibility Levels

LVL 01	**Natural language only** (README)	`"This project requires python 3.10 and numpy."`	❌ Likely unprecise and incomplete. ✅ Create a formal dependency file.
LVL 02	**Manifest file** (requirements.txt)	`requirements.txt:` `pandas`	❌ Installs different versions on different days. ✅ Pin required versions (`pandas==2.1.0`).

Reproducibility Levels

LVL 01	**Natural language only** (README)	`"This project requires python 3.10 and numpy."`	❌ Likely unprecise and incomplete. ✅ Create a formal dependency file.
LVL 02	**Manifest file** (requirements.txt)	`requirements.txt:` `pandas`	❌ Installs different versions on different days. ✅ Pin required versions (`pandas==2.1.0`).
LVL 03	**Top-level versions pinned**	`pandas==2.1.0`	❌ Transitive dependencies still float. ✅ Use a **Lockfile** (`poetry.lock`).

requirements.txt — what is declared

```
django==6.0.2
numpy==2.4.2
pandas==3.0.1
pipdeptree==2.31.0
```

pipdeptree — what actually gets installed

```
Django==6.0.2
├── asgiref [required: >=3.9.1, installed: 3.11.1]
└── sqlparse [required: >=0.5.0, installed: 0.5.5]
pandas==3.0.1
├── numpy [required: >=1.26.0, installed: 2.4.2]
└── python-dateutil [required: >=2.8.2, installed: 2.
  └── six [required: >=1.5, installed: 1.17.0]
pipdeptree==2.31.0
├── packaging [required: >=26, installed: 26.0]
└── pip [required: >=25.2, installed: 26.0.1]
```

uv.lock — what a lockfile captures

```
...
[[package]]
name = "asgiref"
version = "3.11.1"
source = { registry = "https://pypi.org/simple" }
sdist = { url = "https://files.pythonhosted.org/...", h
wheels = [
  { url = "https://files.pythonhosted.org/...", hash =
]
[[package]]
name = "django"
version = "6.0.2"
source = { registry = "https://pypi.org/simple" }
dependencies = [
  { name = "asgiref" },
  { name = "sqlparse" },
  { name = "tzdata", marker = "sys_platform == 'win32'"
]
sdist = { url = "https://files.pythonhosted.org/...", h
wheels = [
  { url = "https://files.pythonhosted.org/...", hash =
]
...
```

Reproducibility Levels

LVL 01	**Natural language only** (README)	`"This project requires python 3.10 and numpy."`	❌ Likely unprecise and incomplete. ✅ Create a formal dependency file.
LVL 02	**Manifest file** (requirements.txt)	`requirements.txt:` `pandas`	❌ Installs different versions on different days. ✅ Pin required versions (`pandas==2.1.0`).
LVL 03	**Top-level versions pinned**	`pandas==2.1.0`	❌ Transitive dependencies still float. ✅ Use a **Lockfile** (`poetry.lock`).
LVL 04	**Dependencies locked**	`poetry.lock`	❌ No system deps (`libblas` , `glibc` , CUDA). ✅ Add **Containers** or **VMs**.

Reproducibility Levels

LVL 01	**Natural language only** (README)	`"This project requires python 3.10 and numpy."`	❌ Likely unprecise and incomplete. ✅ Create a formal dependency file.
LVL 02	**Manifest file** (requirements.txt)	`requirements.txt: pandas`	❌ Installs different versions on different days. ✅ Pin required versions (`pandas==2.1.0`).
LVL 03	**Top-level versions pinned**	`pandas==2.1.0`	❌ Transitive dependencies still float. ✅ Use a **Lockfile** (`poetry.lock`).
LVL 04	**Dependencies locked**	`poetry.lock`	❌ No system deps (`libblas` , `glibc` , CUDA). ✅ Add **Containers** or **VMs**.
LVL 05	**Container environments** (Docker)	`FROM python:3.9`	❌ Base image and apt-get are non-deterministic. ✅ Use declarative specs e.g. **nix**.

Dockerfiles are not reproducible by default

PROBLEMATIC DOCKERFILE

```
FROM python:3.9
RUN apt-get update && apt-get install -y libblas-dev
COPY . .
RUN pip install pandas==2.1.0
```

FROM `python:3.9` is a moving tag — it gets updated silently every time Python releases a patch

RUN `apt-get install libblas-dev` with no version fetches whatever Debian serves today

pip No lockfile — transitive dependencies float freely

IMPROVED DOCKERFILE

```
FROM python:3.9.18@sha256:c3b1...
RUN apt-get update && apt-get install -y \
    libblas-dev=3.11.0
COPY poetry.lock pyproject.toml ./
RUN poetry install --no-root
```

FROM Pinned to an immutable digest — this exact image, forever

RUN `libblas-dev=3.11.0` — explicit system package version

poetry Installs from a lockfile — every transitive dependency is pinned and verified

Docker can be made more reproducible, but it requires deliberate effort.

Reproducibility Levels

LVL 01	**Natural language only** (README)	`"This project requires python 3.10 and numpy."`	❌ Likely unprecise and incomplete. ✅ Create a formal dependency file.
LVL 02	**Manifest file** (requirements.txt)	`requirements.txt:` `pandas`	❌ Installs different versions on different days. ✅ Pin required versions (`pandas==2.1.0`).
LVL 03	**Top-level versions pinned**	`pandas==2.1.0`	❌ Transitive dependencies still float. ✅ Use a **Lockfile** (`poetry.lock`).
LVL 04	**Dependencies locked**	`poetry.lock`	❌ No system deps (`libblas` , `glibc` , CUDA). ✅ Add **Containers** or **VMs**.
LVL 05	**Container environments** (Docker)	`FROM python:3.9`	❌ Base image and apt-get are non-deterministic. ✅ Use declarative specs e.g. **nix**.

LVL 06	**Declarative system env** (nix)	`buildInputs = [`
		`pythonEnv pkgs.git`
		`pkgs.zlib`
		`];`
			❌ Availability of source code of packages.
			✅ Long-term archives: **Software Heritage Foundation**.

```
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable
    flake-utils.url = "github:numtide/flake-utils";
  };
  outputs = { self, nixpkgs, flake-utils }:
    flake-utils.lib.eachDefaultSystem (system:
      let
        pkgs = import nixpkgs { inherit system; };
      in {
        devShells.default = pkgs.mkShell {
          buildInputs = with pkgs; [
            git
            docker-client
            nodejs_25
            python313Packages.uv
            kubectl
          ];
        };
      });
}
```

Declarative System Environment with Nix

Nix is a purely functional package manager. Instead of mutating global state, every package lives in its own isolated path in `/nix/store`.

ADVANTAGES

Reproducible	The same nix file produces identical environments on any machine
Declarative	The entire environment is code and can be version-controlled with your project
No conflicts	Multiple versions of the same tool coexist without clashing
Cross-platform	`eachDefaultSystem` handles different architectures automatically

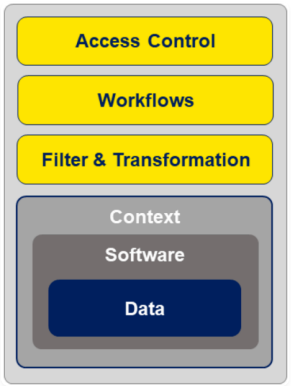
Reproducibility Levels

LVL 01	**Natural language only** (README)	`"This project requires python 3.10 and numpy."`	❌ Likely unprecise and incomplete. ✅ Create a formal dependency file.
LVL 02	**Manifest file** (requirements.txt)	`requirements.txt:` `pandas`	❌ Installs different versions on different days. ✅ Pin required versions (`pandas==2.1.0`).
LVL 03	**Top-level versions pinned**	`pandas==2.1.0`	❌ Transitive dependencies still float. ✅ Use a **Lockfile** (`poetry.lock`).
LVL 04	**Dependencies locked**	`poetry.lock`	❌ No system deps (`libblas` , `glibc` , CUDA). ✅ Add **Containers** or **VMs**.
LVL 05	**Container environments** (Docker)	`FROM python:3.9`	❌ Base image and apt-get are non-deterministic. ✅ Use declarative specs e.g. **nix**.

LVL 06	**Declarative system env** (nix)	`buildInputs = [`
		`pythonEnv pkgs.git`
		`pkgs.zlib`
		`];`
			❌ Availability of source code of packages.
			✅ Long-term archives: **Software Heritage Foundation**.

BEYOND	Long-term archive of packages, hardware environment, sources of non-determinism.		

NFDIxCS — Research Data Management Container & Reusable Execution Environment

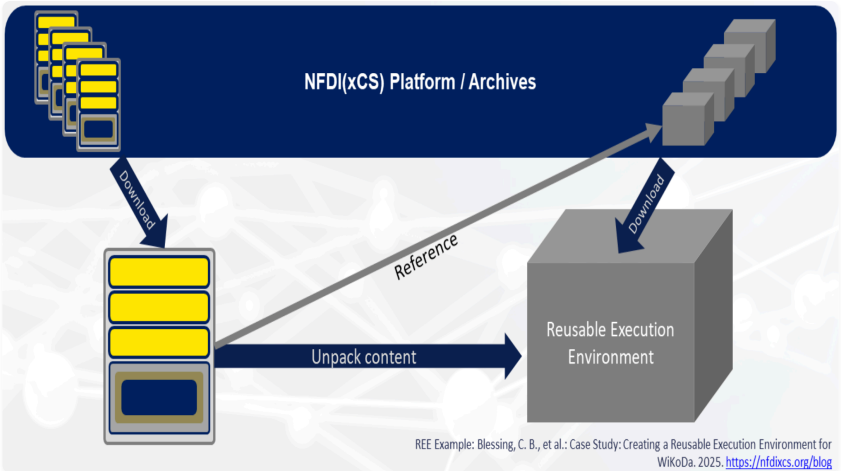


RESEARCH DATA MANAGEMENT CONTAINER (RDMC)

Time capsules for research data and software — bundling data, code, and metadata into a single archived artifact.

- → FAIR by design — metadata always openly accessible
- → Manages access control and workflows
- → Archived for long-term availability

The **Reusable Execution Environment (REE)** is the component inside the RDMC responsible for the executable artifact — unpacked from the container and rebuilt on demand.



A Model for Reusable Execution Environments

FIELDS

`swhid`	SoftWare Hash IDentifier — persistent archive reference of the source code
`source_repository`	URL of the source code repository
`runtime`	Path to a pre-built runtime artifact, if available
`build_runtime_script`	Builds the environment from scratch
`sbom`	Software Bill of Materials — full package inventory
`hardware_description`	CPU, GPU, memory — documents hardware environment
`validation_script`	Verifies rebuilt environment is runnable

EXAMPLES

A Model for Reusable Execution Environments

FIELDS

`swhid`	SoftWare Hash IDentifier — persistent archive reference of the source code
`source_repository`	URL of the source code repository
`runtime`	Path to a pre-built runtime artifact, if available
`build_runtime_script`	Builds the environment from scratch
`sbom`	Software Bill of Materials — full package inventory
`hardware_description`	CPU, GPU, memory — documents hardware environment
`validation_script`	Verifies rebuilt environment is runnable

EXAMPLES

`swhid`	`swh:1:dir:4a6b3c...`

A Model for Reusable Execution Environments

FIELDS

swhid	SoftWare Hash IDentifier — persistent archive reference of the source code
source_repository	URL of the source code repository
runtime	Path to a pre-built runtime artifact, if available
build_runtime_script	Builds the environment from scratch
sbom	Software Bill of Materials — full package inventory
hardware_description	CPU, GPU, memory — documents hardware environment
validation_script	Verifies rebuilt environment is runnable

EXAMPLES

swhid	swh:1:dir:4a6b3c...
source_repository	github.com/org/my-research-software

A Model for Reusable Execution Environments

FIELDS

swhid	SoftWare Hash IDentifier — persistent archive reference of the source code
source_repository	URL of the source code repository
runtime	Path to a pre-built runtime artifact, if available
build_runtime_script	Builds the environment from scratch
sbom	Software Bill of Materials — full package inventory
hardware_description	CPU, GPU, memory — documents hardware environment
validation_script	Verifies rebuilt environment is runnable

EXAMPLES

swhid	swh:1:dir:4a6b3c...
source_repository	github.com/org/my-research-software
runtime	./envs/my-research-env.tar.gz

A Model for Reusable Execution Environments

FIELDS

swhid	SoftWare Hash IDentifier — persistent archive reference of the source code
source_repository	URL of the source code repository
runtime	Path to a pre-built runtime artifact, if available
build_runtime_script	Builds the environment from scratch
sbom	Software Bill of Materials — full package inventory
hardware_description	CPU, GPU, memory — documents hardware environment
validation_script	Verifies rebuilt environment is runnable

EXAMPLES

swhid	swh:1:dir:4a6b3c...
source_repository	github.com/org/my-research-software
runtime	./envs/my-research-env.tar.gz

build_runtime_script

```
#!/bin/bash
docker build \
  -t my-research-env:1.0.0 \
  -f ./Dockerfile \
  --no-cache .
```

A Model for Reusable Execution Environments

FIELDS

swhid	SoftWare Hash IDentifier — persistent archive reference of the source code
source_repository	URL of the source code repository
runtime	Path to a pre-built runtime artifact, if available
build_runtime_script	Builds the environment from scratch
sbom	Software Bill of Materials — full package inventory
hardware_description	CPU, GPU, memory — documents hardware environment
validation_script	Verifies rebuilt environment is runnable

EXAMPLES

swhid	swh:1:dir:4a6b3c...
source_repository	github.com/org/my-research-software
runtime	./envs/my-research-env.tar.gz

build_runtime_script

```
#!/bin/bash
docker build \
  -t my-research-env:1.0.0 \
  -f ./Dockerfile \
  --no-cache .
```

sbom

```
{
  "spdxVersion": "SPDX-2.3",
  "packages": [
```

A Model for Reusable Execution Environments

FIELDS

swhid	SoftWare Hash IDentifier — persistent archive reference of the source code
source_repository	URL of the source code repository
runtime	Path to a pre-built runtime artifact, if available
build_runtime_script	Builds the environment from scratch
sbom	Software Bill of Materials — full package inventory
hardware_description	CPU, GPU, memory — documents hardware environment
validation_script	Verifies rebuilt environment is runnable

EXAMPLES

swhid	swh:1:dir:4a6b3c...
source_repository	github.com/org/my-research-software
runtime	./envs/my-research-env.tar.gz

build_runtime_script

```
#!/bin/bash
docker build \
  -t my-research-env:1.0.0 \
  -f ./Dockerfile \
  --no-cache .
```

sbom

```
{
  "spdxVersion": "SPDX-2.3",
  "packages": [
```

A Model for Reusable Execution Environments

FIELDS

swhid	SoftWare Hash IDentifier — persistent archive reference of the source code
source_repository	URL of the source code repository
runtime	Path to a pre-built runtime artifact, if available
build_runtime_script	Builds the environment from scratch
sbom	Software Bill of Materials — full package inventory
hardware_description	CPU, GPU, memory — documents hardware environment
validation_script	Verifies rebuilt environment is runnable

EXAMPLES

swhid	swh:1:dir:4a6b3c...
source_repository	github.com/org/my-research-software
runtime	./envs/my-research-env.tar.gz

build_runtime_script

```
#!/bin/bash
docker build \
  -t my-research-env:1.0.0 \
  -f ./Dockerfile \
  --no-cache .
```

sbom

```
{
  "spdxVersion": "SPDX-2.3",
  "packages": [
```

Reusable Execution Environment — Services

MODEL

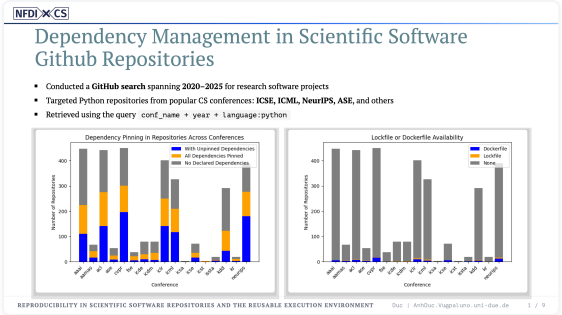
swhid	swh:1:dir:4a6b3c...
source_repository	github.com/org/my-research-software
runtime	./envs/docker-image.tar.gz
build_runtime_script	./scripts/build_env.sh
sbom	./sbom.spdx.json
hardware_description	cpu: Xeon E5 / gpu: A100
validation_script	./scripts/validate_env.sh

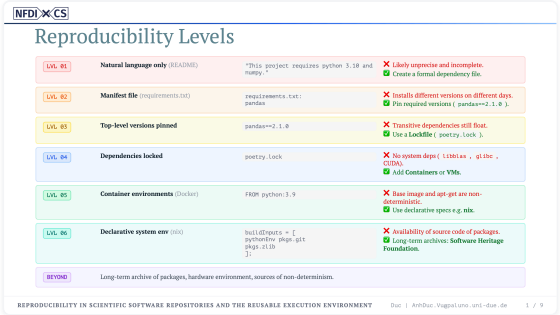
OPERATIONS

- Create REE
- Evaluate Reproducibility Level
- Build Runtime
- Validate Runtime
- Upload to Software Heritage

Conclusion

SUMMARY





FURTHER OPEN CHALLENGES

- → Validating execution environments against hardware requirements
- → Automatically improving reproducibility levels from underspecified environments
- → Build and validation provenance tracking for REE runs

VISIT NFDIxCS AND THE SERVICES

nfdixcs.org/services